人工智能学院学院 大数据管理与分析 课程实验报告

<table>
<tr><td colspan="2">实验题目：实验一　大数据系统基本实验</td><td>学号：202400390057</td></tr>
<tr><td>日期：2026.9.30</td><td>班级：　2 班</td><td>姓名：　李丰润</td></tr>
<tr><td colspan="3">Email：202400390057@mail.sdu.edu.cn</td></tr>
<tr><td colspan="3">实验目的：
1. 熟悉常用的 Linux 操作和 Hadoop 操作：掌握目录与文件管理、文本查看、属性与查找、打包压缩等常用命令，以及启动 Hadoop、在 HDFS 中创建用户目录并上传/下载文件等基本操作；
2. 理解 HDFS 在 Hadoop 体系结构中的角色，熟练使用 HDFS 常用的 Shell 命令，并掌握用 Java API 编程操作 HDFS 的方法；
3. 掌握基本的 MapReduce 编程方法，能够运用 MapReduce 解决数据的合并去重、排序和信息挖掘（祖孙关系）等常见数据处理问题。</td></tr>
<tr><td colspan="3">实验软件和硬件环境：
硬件环境：个人计算机（x86_64 架构）。
软件环境：Oracle VM VirtualBox
Ubuntu 22.04 LTS 64 位操作系统；
JDK 1.8 与 Apache Hadoop 3.3.6；</td></tr>
<tr><td colspan="3">实验原理和方法：
1. Linux 与 Hadoop 基本操作
Hadoop 运行在 Linux 系统上，掌握常用 Linux 命令是使用 Hadoop 的前提。本实验练习了目录与文件操作（cd、ls、mkdir、rmdir、cp、mv、rm）、文本查看（cat、tac、more、head、tail）、文件属性与查找（touch、chown、find）、打包压缩（tar）以及字符串查找（grep）等命令。Hadoop 采用主从架构：HDFS 的 NameNode 负责管理文件系统的命名空间与元数据（文件到数据块的映射），DataNode 存储实际的数据块并周期性地向 NameNode 汇报，SecondaryNameNode 辅助合并编辑日志；伪分布式模式下各守护进程运行在同一台机器（各自独立 JVM）上，便于学习和验证。
2. HDFS 操作
HDFS 通过统一的抽象文件系统 API（org.apache.hadoop.fs.FileSystem）访问，编程入口为 FileSystem.get(new Configuration())。常用操作与 Shell 命令一一对应：上传 copyFromLocalFile/put、下载 copyToLocalFile/get、查看 open/cat、列表 listStatus/ls、创建 mkdirs/create、删除 delete/rm、追加 append/appendToFile、移动 rename/mv，Shell 命令本质上是对 Java API 的封装。HDFS 的设计特点是“一次写入、多次读取”，文件被切块（默认 128 MB，3 副本）分散存储，只支持在文件末尾追加，不支持随机写，更不支持在</td></tr>
</table>

文件开头插入。因此“把内容追加到文件开头”只能采用“读取原内容→重新写回（新内容+原内容）”的方式模拟实现。

3. MapReduce 编程

MapReduce 将计算抽象为 map、shuffle、reduce 三个阶段：map 把输入整理为 (key, value) 中间结果；shuffle 按 key 自动分区、排序并分组；reduce 对每个 key 的 value 列表聚合后输出。本实验三个任务分别利用了它的不同特性：

（1）文件的合并与去重：以整行内容作为 key，MapReduce 对相同 key 天然去重，且输出有序；

（2）数据的排序：以整数作为 key，shuffle 自动升序排序，将 Reducer 数设为 1 可保证全局有序，再用计数器生成排序位次；

（3）祖孙关系挖掘：采用“单表自连接”，把同一张表当作两张表使用，对每条 (child, parent) 记录按 child、parent 分别输出两组带标记的数据，在 reduce 中对同一中间人做笛卡尔积，得到祖孙辈关系。

实验步骤：（不要求罗列完整源代码）

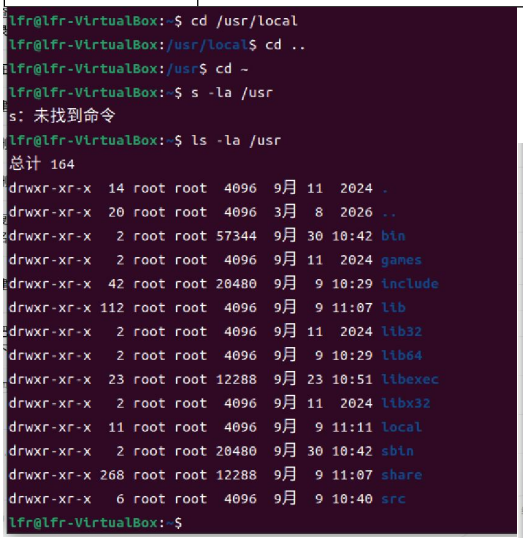
本实验的全部代码（HDFSApp.java、MergeDedup.java、Sort.java、STJoin.java 及 README.md）已整理上传至 GitHub 公开仓库，仓库链接：https://github.com/<GitHub 用户名>/bigdata-lab1。

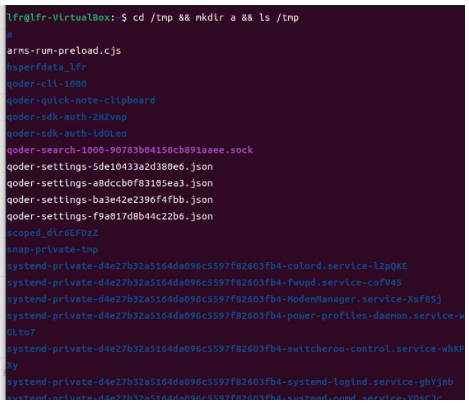
步骤一　Linux 常用命令操作

以 hadoop 用户登录 Linux 系统，打开终端，按实验要求逐条练习常用命令，代表性操作如下：

类别	命令示例	说明
目录切换与查看	cd /usr/local、cd ..、cd ~、ls -la /usr	切换目录、查看目录内容
目录创建与删除	mkdir a、mkdir -p a1/a2/a3/a4、rmdir /tmp/a、rmdir -p a1/a2/a3/a4	创建目录、递归创建、删除空目录
复制/移动/删除	cp ~/.bashrc /usr/bashrc1、cp -r /tmp/test /usr、mv /usr/bashrc1 /usr/test/、mv /usr/test /usr/test2、rm /usr/test2/bashrc1、rm -r /usr/test2	复制并改名、移动与更名、删除文件与目录
文本查看	cat、tac、more；head -n 20、head -n -50；tail -n 20、tail -n +51 ~/.bashrc	正序/逆序、分页查看；取前 20 行、去掉末尾 50 行、取末尾 20 行、从第 51 行开始显示
属性与查找	touch /tmp/hello、touch -d "5 days ago" /tmp/hello、chown root /tmp/hello、find ~ -name .bashrc	创建空文件并查看时间、把时间改为 5 天前、将属主改为 root、按名称查找文件
打包与字符	tar -czf /test.tar.gz -C / test、tar	打包压缩、解压到 /tmp、

串查找	-xzf /test.tar.gz -C /tmp、grep examples ~/.bashrc	在文件中查找字符串





示例截图

步骤二　启动 Hadoop 并完成 HDFS 基本操作

```
start-dfs.sh && start-yarn.sh                          # 启动 HDFS 与 YARN；jps 可查看各守护进程
hdfs dfs -mkdir -p /user/hadoop                        # 为 hadoop 用户创建 HDFS 用户目录
hdfs dfs -mkdir /user/hadoop/test                      # 创建 test 文件夹，并 ls 查看文件列表
hdfs dfs -put ~/.bashrc /user/hadoop/test              # 将本地 ~/.bashrc 上传到 HDFS 的 test 文件夹
hdfs dfs -get /user/hadoop/test /usr/local/hadoop/     # 将 HDFS 的 test 文件夹下载到本地
```

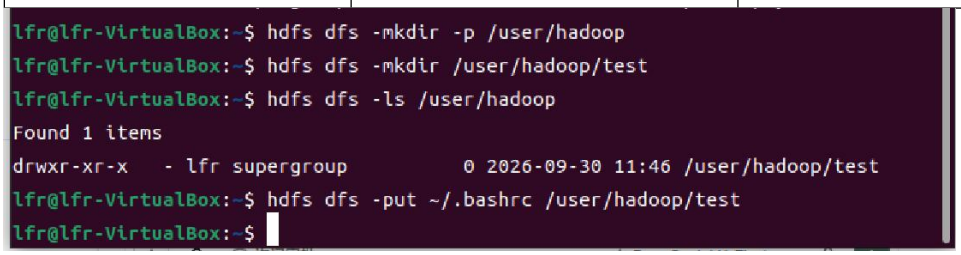
执行后 HDFS 的 /user/hadoop/test 下出现 .bashrc 文件，表明上传成功；下载命令把 test 文件夹复制到本地 /usr/local/hadoop 目录下。

步骤三　HDFS 操作（Shell 命令 + Java API）

分别用 HDFS 的 Shell 命令和 Java API 实现题目要求的 10 个功能，两种方式结果一致。Shell 命令的关键指令与运行结果如下：

功能	**Shell 命令**	**运行结果**
上传（已存在时选择追加/覆盖）	put；appendToFile；put -f	首次上传、追加、覆盖三种情况均正确
下载（同名自动重命名）	get + 本地改名	原文件未被覆盖，得到副本（.bak）
输出文件内容到终端	hdfs dfs -cat	正确输出文件内容
显示文件信息	hdfs dfs -ls / -stat	显示权限、大小、修改时间、路径
递归显示目录信息	hdfs dfs -ls -R	递归列出全部文件与子目录
创建/删除文件（父目录	mkdir -p + touchz；rm	自动创建父目录并完成创

自动创建）		建、删除
创建/删除目录（仅空目录可删除）	mkdir -p；rmdir	空目录删除成功；非空目录报错拒删
追加到开头/结尾	appendToFile；下载拼接后 put -f	依次得到“原内容→末尾追加→开头+原+尾”三种状态
删除指定文件	hdfs dfs -rm	删除成功
移动文件	hdfs dfs -mv	移动成功



Java API 部分编写了菜单式程序 HDFSApp（源码见仓库 part2_hdfs/HDFSApp.java），用 FileSystem API 实现上述全部功能，编译与运行命令为：

```
javac -classpath "$(hadoop classpath)" HDFSApp.java
java  -classpath ".:$(hadoop classpath)" HDFSApp
```

运行结果表明 10 个功能均正确实现。

步骤四　MapReduce 初级编程

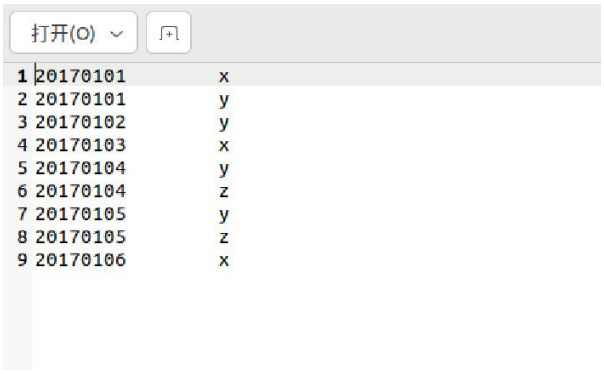
三个程序均基于 org.apache.hadoop.mapreduce 编写（源码见仓库 part3_mapreduce/），编译、打包与提交运行的命令示例：

```
javac -classpath "$(hadoop classpath)" -d . MergeDedup.java Sort.java
STJoin.java   # 编译三个程序
jar cf MergeDedup.jar MergeDedup*.class     # 打包，Sort.jar、STJoin.jar 同理
hadoop jar MergeDedup.jar MergeDedup /input/merge /output/merge    # 提交到
YARN 运行
hdfs dfs -cat /output/merge/*               # 查看输出（注意：输出目录必须事先不
存在）
```

（1）文件的合并和去重。map 把整行内容作为 key 输出，reduce 对每个 key 只输出一次，自动完成去重并保持有序，核心代码：

```
context.write(new Text(line), new Text(""));      // map：整行内容作为 key
context.write(key, new Text(""));                   // reduce：每个 key 只输出
一次
```

将输入文件 A、B 上传后运行，输出共 9 行：重复记录（20170102 y、20170103 x、20170105 y 等）被剔除，内容按 key 升序排列，与题目样例一致：



（2）数据的排序。map 以整数为 key 输出，shuffle 自动升序排序；设置 job.setNumReduceTasks(1) 保证全局有序，reduce 中用计数器生成排序位次，核心代码：

```
job.setNumReduceTasks(1);                                    // 单 Reducer，保证全局有序
counter++; context.write(new IntWritable(counter), key);    // 生成位次并输出
```

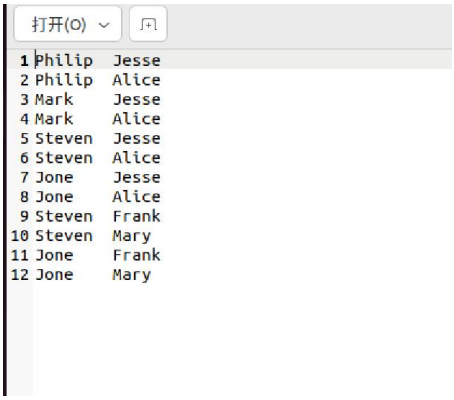
运行后输出 11 行“位次 整数”，位次从 1 连续递增，与题目样例一致：



（3）祖孙辈关系挖掘。采用单表自连接：对每条 (child, parent) 记录，以 child 为 key 输出“1”+parent（该 key 的父辈），以 parent 为 key 输出“2”+child（该 key 的子辈），核心代码：

```
context.write(new Text(child),  new Text("1" + parent));  // 打标记：父辈
context.write(new Text(parent), new Text("2" + child));   // 打标记：子辈
```

reduce 中对同一 key（中间人）将“子辈×父辈”做笛卡尔积得到祖孙关系。运行后输出 12 条祖孙关系。



结论分析与体会：

1. 通过本实验，我系统练习了 Linux 常用命令和 Hadoop/HDFS 基本操作，理解了 HDFS“NameNode 管元数据、DataNode 存数据块”的主从架构，掌握了用 Shell 命令和 Java API 两种方式操作 HDFS 的方法，并体会到二者的一一对应关系。

2. 在 MapReduce 部分，我体会到“key 的设计”是编程的核心：数据去重与排序都依赖 shuffle 对 key 的排序与分组能力；关系挖掘则通过给同一记录打标记、对同一 key 下不同角色的数据做笛卡尔积来实现。将 Reducer 数设为 1 可使输出全局有序，但会牺牲并行度，实际应用需在正确性与性能之间权衡。

3. 实验中遇到并解决了若干问题：① 重新格式化 NameNode 后 DataNode 无法启动，日志提示 Incompatible clusterIDs（两侧 clusterID 不一致），删除 DataNode 数据目录后重启即恢复；② 以普通用户身份向 /home/hadoop 写文件时报“权限不够”，改将测试文件放到自己有权访问的目录后上传即解决；③ HDFS 不支持在文件头部插入内容，通过“读取原内容、重新写回”的方式模拟实现。

4. 体会：大数据实验对环境依赖较强，遇到问题时应优先查看 NameNode、DataNode 等日志定位根因，而不是盲目重装；在低配置机器上合理调低各组件内存参数，也是实验顺利完成的保障。

就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1—3 道问答题：

问题 1：向 HDFS 上传文件时，如果指定的文件在 HDFS 中已经存在，如何做到由用户来指定“追加到原有文件末尾”还是“覆盖原有的文件”？

答：Shell 方式下，hdfs dfs -put 默认不允许覆盖已存在的文件：选择

“追加”可在上传前执行 hdfs dfs -appendToFile 本地文件 HDFS 文件，把本地文件的内容追加到 HDFS 文件末尾；选择“覆盖”则使用 hdfs dfs -put -f 本地文件 目标路径（-f 表示强制覆盖）。Java API 方式下，先用 fs.exists() 判断目标是否已存在：选择追加时调用 fs.append() 获得输出流写入，选择覆盖时调用 fs.create(dst, true)（overwrite 参数为 true）重新写入。HDFSApp 程序即按上述逻辑实现。

问题 2：排序程序中为什么要把 Reduce 任务数设置为 1？

答：MapReduce 默认按 key 的哈希把中间结果分配给多个 Reducer，各 Reducer 内部有序，但多个输出文件之间整体无序。要得到全局升序的结果、并生成从 1 开始连续递增的位次，必须让所有 key 进入同一个 Reducer，因此设置 job.setNumReduceTasks(1)。代价是并行度降低；数据量大时可改用“自定义分区 + 全排序”在保证有序的同时提高并行度。

问题 3：HDFS 无法直接向文件开头插入内容，本实验是如何实现“追加到开头”的？

答：HDFS 面向“一次写入、多次读取”的大文件场景，文件被切分成数据块分散存储在多个 DataNode 上；若允许从文件开头插入内容，后面所有数据块都需要重写和搬迁，并同步更新 NameNode 元数据，代价极高，因此 HDFS 只支持在文件末尾追加。本实验采用“读取原内容→重新写回（新内容+原内容）”的方式模拟实现：在 Java API 中，先读取原文件全部字节，再删除原文件并重建，依次写入新内容与原内容；Shell 方式下可采用“下载→本地合并→上传覆盖”的思路。